

There cases where a thread wishes to check whether a condition is true before continuing its execution.

For example, a parent thread might wish to check whether a child thread has completed before continuing (this is called `join()`).

How should such a wait be implemented?

```
1 void *child(void *arg) {
2     printf("child\n");
3     // XXX how to indicate we are done?
4     return NULL;
5 }
6
7 int main(int argc, char *argv[]) {
8     printf("parent: begin\n");
9     pthread_t c;
10    Pthread_create(&c, NULL, child, NULL); // child
11    // XXX how to wait for child?
12    printf("parent: end\n");
13    return 0;
14 }
```

We could use a shared variable as below. This solution will generally work, but it is hugely inefficient as the parent spins and wastes CPU time.

```
1 volatile int done = 0;
2
3 void *child(void *arg) {
4     printf("child\n");
5     done = 1;
6     return NULL;
7 }
8
9 int main(int argc, char *argv[]) {
10    printf("parent: begin\n");
11    pthread_t c;
12    Pthread_create(&c, NULL, child, NULL); // child
13    while (done == 0)
14        ; // spin
15    printf("parent: end\n");
16    return 0;
17 }
```

What we want is to put parent to sleep until the condition we are waiting for comes true.

THE CRUX: HOW TO WAIT FOR A CONDITION

In multi-threaded programs, it is often useful for a thread to wait for some condition to become true before proceeding. The simple approach, of just spinning until the condition becomes true, is grossly inefficient and wastes CPU cycles, and in some cases, can be incorrect. Thus, how should a thread wait for a condition?

Definition and Routines

To wait for condition to become true, a thread can use a **condition variable**.

A condition variable is an explicit queue that threads can put themselves on when some state of execution (i.e. some **condition**) is not as desired by waiting.

Some other thread, when it changes state, can wake one or more of those waiting threads and thus allow them to continue (by **signaling** on condition).

Declare such a variable as: `pthread_cond_t c;`

A condition variable has two associated parameters:

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
```

This puts the calling thread to sleep

It also takes a mutex as a parameter, & assumes mutex is locked when it is called.

It must release the lock & put the calling thread to sleep atomically.

Further, when the thread wakes up, it must reacquire the lock before returning,

this is to prevent race conditions when

a thread is trying put itself to sleep.

```
pthread_cond_signal(pthread_cond_t *c);
```

This wakes a sleeping thread.

```
1  int done = 0;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5  void thr_exit() {
6      Pthread_mutex_lock(&m);
7      done = 1;
8      Pthread_cond_signal(&c);
9      Pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13     printf("child\n");
14     thr_exit();
15     return NULL;
16 }
17
18 void thr_join() {
19     Pthread_mutex_lock(&m);
20     while (done == 0)
21         Pthread_cond_wait(&c, &m);
22     Pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26     printf("parent: begin\n");
27     pthread_t p;
28     Pthread_create(&p, NULL, child, NULL);
29     thr_join();
30     printf("parent: end\n");
31     return 0;
32 }
```

There are two cases to consider:

1. Parent creates child, and continues to run. It calls `thr_join()`.

It acquires the lock & finds `done=0`, and puts itself to sleep by calling `wait()` & release the lock.

The child will run, calls `thr_exit()` & acquires lock, sets `done = 1`, signals parent to wake up & release the lock.

The parent returns from `wait()` after reacquiring the lock, releases it & runs to completion.

2. The child runs immediately on creation, sets `done` to 1 & tries to wake a sleeping thread but finds none & simply returns.

The parent calls `thr_exit()`, sees `done = 1`, returns & runs to completion.

Lets try alternate implementation:

1. Do we need the `done` variable? What if the code looked like below?

```
1 void thr_exit() {  
2     Pthread_mutex_lock(&m);  
3     Pthread_cond_signal(&c);  
4     Pthread_mutex_unlock(&m);  
5 }  
6  
7 void thr_join() {  
8     Pthread_mutex_lock(&m);  
9     Pthread_cond_wait(&c, &m);  
10    Pthread_mutex_unlock(&m);  
11 }
```

lets imagine a case where child runs first. It will signal, but find none are sleeping & will finish.

Now, when parent runs, it will call `thr_join()` & simply sleep forever.

From this example, we see that some sort of done variable is needed.

2. Do we really need to acquire lock in `thr_exit()` & `thr_wait()`? What if code looked like below:

```
1 void thr_exit() {
2     done = 1;
3     Pthread_cond_signal(&c);
4 }
5
6 void thr_join() {
7     if (done == 0)
8         Pthread_cond_wait(&c);
9 }
```

This implementation is for understanding & doesn't actually exist.

Lets imagine a case where parent runs first & finds `done=0` & then is interrupted.

The child then runs, sets `done=1`, signals a sleeping thread to wake but finds none & simply finishes.

The parent comes back & sleeps forever.

TIP: ALWAYS HOLD THE LOCK WHILE SIGNALING

Although it is strictly not necessary in all cases, it is likely simplest and best to hold the lock while signaling when using condition variables. The example above shows a case where you *must* hold the lock for correctness; however, there are some other cases where it is likely OK not to, but probably is something you should avoid. Thus, for simplicity, **hold the lock when calling signal**.

The converse of this tip, i.e., hold the lock when calling wait, is not just a tip, but rather mandated by the semantics of wait, because wait always (a) assumes the lock is held when you call it, (b) releases said lock when putting the caller to sleep, and (c) re-acquires the lock just before returning. Thus, the generalization of this tip is correct: **hold the lock when calling signal or wait**, and you will always be in good shape.

The Producer/Consumer (Bounded Buffer) Problem

Imagine one or more producer threads & one or more consumer threads.

Producers generate data items, and place them in a buffer.

Consumers grab said items from the buffer and consume them in some way.

This arrangement occurs in many real systems:

- A multi-thread web server, a producer puts HTTP request into a work queue; consumer threads take requests out of this queue & process them.
- grep foo file.txt | wc -l.
producer consumer

Producer writes "foo" from file.txt to pipe & consumer consumes them as counts.

As bounded buffer is a shared resource, we require synchronization.

Lets assume a single integer space as our buffer for simplicity.

```
1  int buffer;  
2  int count = 0; // initially, empty  
3  
4  void put(int value) {  
5      assert(count == 0);  
6      count = 1;  
7      buffer = value;  
8  }  
9  
10 int get() {  
11     assert(count == 1);  
12     count = 0;
```

```
13     return buffer;  
14 }
```

`put()` routine assumes buffer is empty by assertion, then simply puts value into buffer & marks it full by setting `count=1`.

`get()` routine assumes buffer is full by assertion, then simply returns it & sets it to empty by `count=0`.

Our condition variable will be `count`, only put data in buffer when `count=0` & only get data from buffer when `count=1`.

```
1  void *producer(void *arg) {  
2      int i;  
3      int loops = (int) arg;  
4      for (i = 0; i < loops; i++) {  
5          put(i);  
6      }  
7  }  
8  
9  void *consumer(void *arg) {  
10     while (1) {  
11         int tmp = get();  
12         printf("%d\n", tmp);  
13     }  
14 }
```

This work is to be done by consumer & producer threads.

Above code shows code for producer that puts an integer into buffer loop times, and a consumer that gets data out of buffer & prints it (forever).

A broken solution

Imagine one consumer & one producer thread.

```
1  int loops; // must initialize somewhere...
2  cond_t  cond;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);           // p1
9          if (count == 1)                       // p2
10             Pthread_cond_wait(&cond, &mutex); // p3
11          put(i);                               // p4
12          Pthread_cond_signal(&cond);          // p5
13          Pthread_mutex_unlock(&mutex);        // p6
14      }
15  }
16
17  void *consumer(void *arg) {
18      int i;
19      for (i = 0; i < loops; i++) {
20          Pthread_mutex_lock(&mutex);           // c1
21          if (count == 0)                       // c2
22             Pthread_cond_wait(&cond, &mutex); // c3
23          int tmp = get();                      // c4
24          Pthread_cond_signal(&cond);          // c5
25          Pthread_mutex_unlock(&mutex);        // c6
26          printf("%d\n", tmp);
27      }
28  }
```

When producer wants to fill the buffer, it waits for it to be empty ($p_1 - p_3$).

The consumer has the exact same logic, but waits for a different condition: fullness ($c_1 - c_3$).

What if we had more of these threads (e.g., two consumers)?

1. Assume there are two consumers (T_{c1}, T_{c2}) & one producer (T_p).

First, a consumer T_{c1} runs & acquires the lock (c_1), finds $\text{count}=0$ (c_2), and waits (c_3) which releases the lock.

Then, producer (T_p) runs. It acquires the lock (p_1), finds $\text{count}=0$ (p_2), fills the buffer (p_4) & signals the consumers to move from sleep state to ready state.

The producer then continues to run & finds $\text{count}=1$ & sleeps (p_6, p_1-p_3)

Now, consumer T_{c2} runs & consumes the value from buffer & sets $\text{count} = 0$ (c_1-2, c_4-6)

The consumer T_{c1} runs & calls $\text{get}()$ (c_4) but as $\text{count}=0$, our code asserts.

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Run		Ready		Ready	0	
c2	Run		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep		Ready	p1	Run	0	
	Sleep		Ready	p2	Run	0	
	Sleep		Ready	p4	Run	1	Buffer now full
	Ready		Ready	p5	Run	1	T_{c1} awoken
	Ready		Ready	p6	Run	1	
	Ready		Ready	p1	Run	1	
	Ready		Ready	p2	Run	1	
	Ready		Ready	p3	Sleep	1	Buffer full; sleep
	Ready	c1	Run		Sleep	1	T_{c2} sneaks in ...
	Ready	c2	Run		Sleep	1	
	Ready	c4	Run		Sleep	0	... and grabs data
	Ready	c5	Run		Ready	0	T_p awoken
	Ready	c6	Run		Ready	0	
c4	Run		Ready		Ready	0	Oh oh! No data

Mesa semantics:

When a thread is signaled, it's only a hint that something has changed, but there is no guarantee that when the woken thread runs, the state will be as desired.

Hoare Semantics:

It is harder to build, but provides a stronger guarantee that woken thread will run immediately upon being woken up.

Better, But Still Broken: While, not if.

change the if to a while. Think about why this works; now consumer T_{c1} wakes up and (with the lock held) immediately re-checks the state of the shared variable ($c2$). If the buffer is empty at that point, the consumer simply goes back to sleep ($c3$). The corollary if is also changed to a while in the producer ($p2$). Thanks to Mesa semantics, a simple rule to remember with condition variables is to **always use while loops**.

However, we haven't addressed the second problem.

2. T_{c1} runs, finds $\text{count}=0$ & sleeps ($c1-c3$). Then, T_{c2} runs, finds $\text{count}=0$ & sleeps as well ($c1-c3$).

Now, T_p runs, finds $\text{count}=0$ & fills the buffer & signals a thread to wake up ($P1-P2, P4-P6$), T_{c1} wakes up.

It continues to run, finds $\text{count}=1$ & sleeps ($P1-P3$).

Now, T_{c1} runs & consumes value from buffer & signals a thread to wake up ($C4-C6$), T_{c2} wakes up.

It continues to run, finds $\text{count}=0$, & sleeps ($c1-c3$).

T_{c2} runs, finds $\text{count}=0$ & sleeps ($c1-c3$).

Now, every thread is sleeping.

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Run		Ready		Ready	0	Nothing to get
c2	Run		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	
	Sleep	c1	Run		Ready	0	Nothing to get
	Sleep	c2	Run		Ready	0	
	Sleep	c3	Sleep		Ready	0	
	Sleep		Sleep	p1	Run	0	

	Sleep		Sleep	p2	Run	0	
	Sleep		Sleep	p4	Run	1	Buffer now full
	Ready		Sleep	p5	Run	1	T_{c1} awoken
	Ready		Sleep	p6	Run	1	
	Ready		Sleep	p1	Run	1	
	Ready		Sleep	p2	Run	1	
	Ready		Sleep	p3	Sleep	1	Must sleep (full)
c2	Run		Sleep		Sleep	1	Recheck condition
c4	Run		Sleep		Sleep	0	T_{c1} grabs data
c5	Run		Ready		Sleep	0	Oops! Woke T_{c2}
c6	Run		Ready		Sleep	0	
c1	Run		Ready		Sleep	0	
c2	Run		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Run		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep...

The Single Buffer Producer/Consumer Solution

Use two condition variables in order to properly signal which type of thread should wake up when the state of the system changes.

```

1  cond_t  empty, fill;
2  mutex_t mutex;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          Pthread_mutex_lock(&mutex);
8          while (count == 1)
9              Pthread_cond_wait(&empty, &mutex);
10         put(i);
11         Pthread_cond_signal(&fill);
12         Pthread_mutex_unlock(&mutex);
13     }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         Pthread_mutex_lock(&mutex);
20         while (count == 0)
21             Pthread_cond_wait(&fill, &mutex);
22         int tmp = get();
23         Pthread_cond_signal(&empty);
24         Pthread_mutex_unlock(&mutex);
25         printf("%d\n", tmp);
26     }
27 }
```

In above code, producer threads wait on the condition empty, and signals fill.

Conversely, consumer threads wait on fill & signal empty.

This solves the second problem as a consumer thread can never accidentally wake a consumer, a producer can never accidentally wake a producer.

The Correct Producer/Consumer Solution

The last change we need to make to enable more concurrency is to add more buffer slots.

The producer only sleeps if all buffers are filled (p_2) & consumer only sleeps if all buffers are empty (c_2).

```
1  int buffer[MAX];
2  int fill_ptr = 0;
3  int use_ptr  = 0;
4  int count    = 0;
5
6  void put(int value) {
7      buffer[fill_ptr] = value;
8      fill_ptr = (fill_ptr + 1) % MAX;
9      count++;
10 }
11
12 int get() {
13     int tmp = buffer[use_ptr];
14     use_ptr = (use_ptr + 1) % MAX;
15     count--;
16     return tmp;
17 }
```

Figure 30.13: The Correct Put And Get Routines

```
1  cond_t empty, fill;
2  mutex_t mutex;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          Pthread_mutex_lock(&mutex);           // p1
8          while (count == MAX)                  // p2
9              Pthread_cond_wait(&empty, &mutex); // p3
10         put(i);                                // p4
11         Pthread_cond_signal(&fill);            // p5
12         Pthread_mutex_unlock(&mutex);          // p6
13     }
14 }
```



```

15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         Pthread_mutex_lock(&mutex);           // c1
20         while (count == 0)                     // c2
21             Pthread_cond_wait(&fill, &mutex); // c3
22         int tmp = get();                       // c4
23         Pthread_cond_signal(&empty);          // c5
24         Pthread_mutex_unlock(&mutex);         // c6
25         printf("%d\n", tmp);
26     }
27 }

```

Figure 30.14: The Correct Producer/Consumer Synchronization

TIP: USE WHILE (NOT IF) FOR CONDITIONS

When checking for a condition in a multi-threaded program, using a `while` loop is always correct; using an `if` statement only might be, depending on the semantics of signaling. Thus, always use `while` and your code will behave as expected.

Using `while` loops around conditional checks also handles the case where **spurious wakeups** occur. In some thread packages, due to details of the implementation, it is possible that two threads get woken up though just a single signal has taken place [L11]. Spurious wakeups are further reason to re-check the condition a thread is waiting on.

when a thread wakes up from a `wait()` without receiving a signal.

Covering Conditions

```

1 // how many bytes of the heap are free?
2 int bytesLeft = MAX_HEAP_SIZE;
3
4 // need lock and condition too
5 cond_t c;
6 mutex_t m;
7
8 void *
9 allocate(int size) {
10     Pthread_mutex_lock(&m);
11     while (bytesLeft < size)
12         Pthread_cond_wait(&c, &m);
13     void *ptr = ...; // get mem from heap
14     bytesLeft -= size;
15     Pthread_mutex_unlock(&m);
16     return ptr;
17 }
18

```



```

19 void free(void *ptr, int size) {
20     Pthread_mutex_lock(&m);
21     bytesLeft += size;
22     Pthread_cond_signal(&c); // whom to signal??
23     Pthread_mutex_unlock(&m);
24 }

```

The problem is illustrated by an example of a simple multi-threaded memory allocation library.

In above code, when a thread calls `allocate()`, it might have to wait in order for more memory to become free.

Conversely, when a thread frees memory, it signals more memory to become free.

However, which waiting thread (there can be more than one) to wake up?

Consider the following scenario. Assume there are zero bytes free.

thread T_a calls `allocate(100)`, followed by thread T_b which asks for less memory by calling `allocate(10)`.

Both T_a & T_b thus wait on the condition & sleep.

At that point, thread T_c calls `free(50)`. Unfortunately, when it calls `signal` to wake a waiting thread, it might not wake the correct waiting thread T_b .

The solution suggested was to replace

`pthread_cond_signal()` by `pthread_cond_broadcast()`, which wakes all waiting threads up.

The downside is negative performance impact, as we might be needlessly wake up many other waiting threads that shouldn't yet be awake. Those threads will simply wake up, recheck the condition, and then go to sleep.

Such a condition is called a *covering condition*.

We can also use this solution in single condition variable produces/consumers problem. However, a better solution existed.